

Lab 15 - Python Threat Intelligence API Automation

AbuseIPDB Enrichment, Risk Classification, and JSON/CSV Reporting

Objective: Develop a Python security automation tool that validates a public IP address, queries the AbuseIPDB REST API, parses the returned JSON data, assigns a lab-defined risk level, recommends an analyst action, and automatically saves structured JSON and CSV output.

Environment

Component	Purpose
Windows host	Python development and controlled testing environment
Python	Implements validation, API requests, parsing, classification, and reporting
requests library	Sends HTTPS GET requests to AbuseIPDB
AbuseIPDB API	Provides IP reputation and report data
Environment variable	Stores the API key outside the source code
JSON / CSV	Stores analyst-friendly output for automation and reporting

Lab Workspace

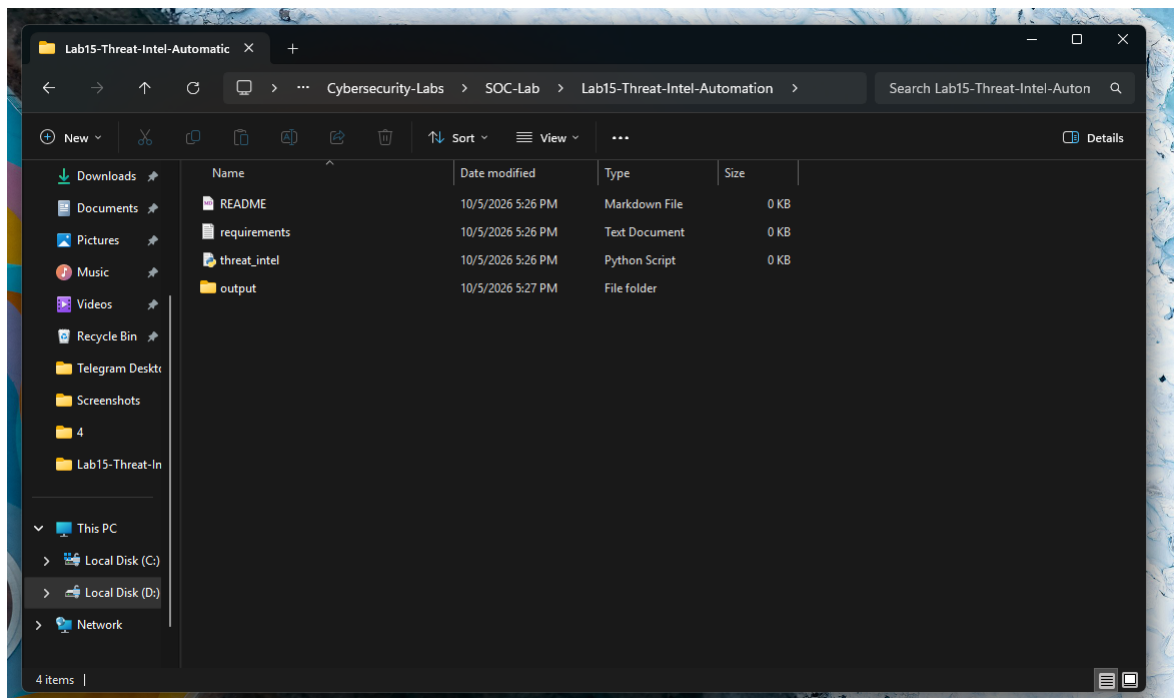


Figure 1 - Lab 15 folder structure containing the Python script, requirements file, README, and output directory.

API Integration

The script loads the AbuseIPDB API key from the Windows environment variable **ABUSEIPDB_API_KEY**, preventing the credential from being hard-coded into the script or repository. The script uses the AbuseIPDB **/api/v2/check** endpoint with a 90-day reporting window and verbose output.

```
headers = {
    "Accept": "application/json",
    "Key": API_KEY
}

params = {
    "ipAddress": ip,
    "maxAgeInDays": 90,
    "verbose": True
}

response = requests.get(
    API_URL,
    headers=headers,
    params=params,
    timeout=10
)
```

Initial API Validation

A public test IP was submitted to confirm successful connectivity, authentication, JSON parsing, and access to the expected AbuseIPDB fields.

```
D:\Cybersecurity-Labs\SOC-Lab\Lab15-Threat-Intel-Automation>python threat_intel.py
=== Threat Intelligence IP Checker ===
Enter an IP address: 118.25.6.39
{
  "data": {
    "ipAddress": "118.25.6.39",
    "isPublic": true,
    "ipVersion": 4,
    "isWhitelisted": false,
    "abuseConfidenceScore": 7,
    "countryCode": "CN",
    "usageType": "Data Center/Web Hosting/Transit",
    "isp": "Tencent Cloud Computing (Beijing) Co., Ltd",
    "domain": "tencentcloud.com",
    "hostnames": [],
    "isTor": false,
    "countryName": "China",
    "totalReports": 2,
    "numDistinctUsers": 2,
    "lastReportedAt": "2026-09-17T22:17:21+00:00",
    "reports": [
      {
        "reportedAt": "2026-09-17T22:17:21+00:00",
        "comment": "Honeypot: SSH brute force",
        "categories": [
          18,
          22
        ],
        "reporterId": 350015,
        "reporterCountryCode": "US",
        "reporterCountryName": "United States of America"
      },
      {
        "reportedAt": "2026-08-31T18:08:32+00:00",
        "comment": "CSF test",
        "categories": [
          18,
          22
        ],
        "reporterId": 344099,
        "reporterCountryCode": "US",
        "reporterCountryName": "United States of America"
      }
    ]
  }
}
```

Figure 2 - Raw AbuseIPDB JSON response returned successfully by the Python script.

Risk Classification and Analyst Output

The script converts the AbuseIPDB confidence score into a simple lab-defined risk level and corresponding analyst action. These thresholds are for this portfolio lab and are **not official AbuseIPDB severity categories**.

Abuse confidence score	Risk level / Analyst action
0-24	LOW - Document and monitor
25-74	MEDIUM - Review additional context and investigate
75-100	HIGH - Escalate for analyst investigation

Successful Public-IP Lookup

```
D:\Cybersecurity-Labs\SOC-Lab\Lab15-Threat-Intel-Automation>python threat_intel.py
=== Threat Intelligence IP Checker ===
Enter an IP address: 118.25.6.39

=== Threat Intelligence Result ===
IP Address: 118.25.6.39
Country: CN
ISP: Tencent Cloud Computing (Beijing) Co., Ltd
Domain: tencentcloud.com
Usage Type: Data Center/Web Hosting/Transit
Abuse Confidence Score: 7
Total Reports: 2
Last Reported: 2026-09-17T22:17:21+00:00
Risk Level: LOW
Recommended Action: Document and monitor
Report Saved: output/118.25.6.39_report.json
CSV Updated: output/threat_intel_results.csv
```

Figure 3 - Clean analyst-style output including risk level, recommended action, and saved report locations.

Field	Observed result
IP address	118.25.6.39
Country	CN
ISP	Tencent Cloud Computing (Beijing) Co., Ltd
Domain	tencentcloud.com
Abuse confidence score	7
Risk level	LOW
Recommended action	Document and monitor

Input Validation

Before making an external API request, the script validates the supplied value with Python's **ipaddress** module. Invalid, private, loopback, reserved, multicast, or unspecified addresses are rejected so that unnecessary or inappropriate API requests are avoided.

```
D:\Cybersecurity-Labs\SOC-Lab\Lab15-Threat-Intel-Automation>python threat_intel.py
=== Threat Intelligence IP Checker ===
Enter an IP address: 999.999.999.999
Invalid, private, or reserved IP address.

D:\Cybersecurity-Labs\SOC-Lab\Lab15-Threat-Intel-Automation>python threat_intel.py
=== Threat Intelligence IP Checker ===
Enter an IP address: 192.168.1.10
Invalid, private, or reserved IP address.

D:\Cybersecurity-Labs\SOC-Lab\Lab15-Threat-Intel-Automation>python threat_intel.py
=== Threat Intelligence IP Checker ===
Enter an IP address: 118.25.6.39

=== Threat Intelligence Result ===
IP Address: 118.25.6.39
Country: CN
ISP: Tencent Cloud Computing (Beijing) Co., Ltd
Domain: tencentcloud.com
Usage Type: Data Center/Web Hosting/Transit
Abuse Confidence Score: 7
Total Reports: 2
Last Reported: 2026-09-17T22:17:21+00:00
Risk Level: LOW
Recommended Action: Document and monitor
Report Saved: output/118.25.6.39_report.json
CSV Updated: output/threat_intel_results.csv
```

Figure 4 - Final validation tests showing invalid, private, and valid public-IP handling.

Test input	Expected / observed behavior
999.999.999.999	Rejected - invalid IP address
192.168.1.10	Rejected - private IP address
118.25.6.39	Accepted - API enrichment, classification, and reporting completed

Validation Result

The final script successfully distinguished invalid and private inputs from a valid public IP and only performed the AbuseIPDB lookup for the valid public address.

Automated JSON and CSV Reporting

For each successful lookup, the script writes a per-IP JSON report and appends the result to a cumulative CSV file. This allows the data to be consumed by other scripts, spreadsheets, SIEM tooling, or future automation workflows.

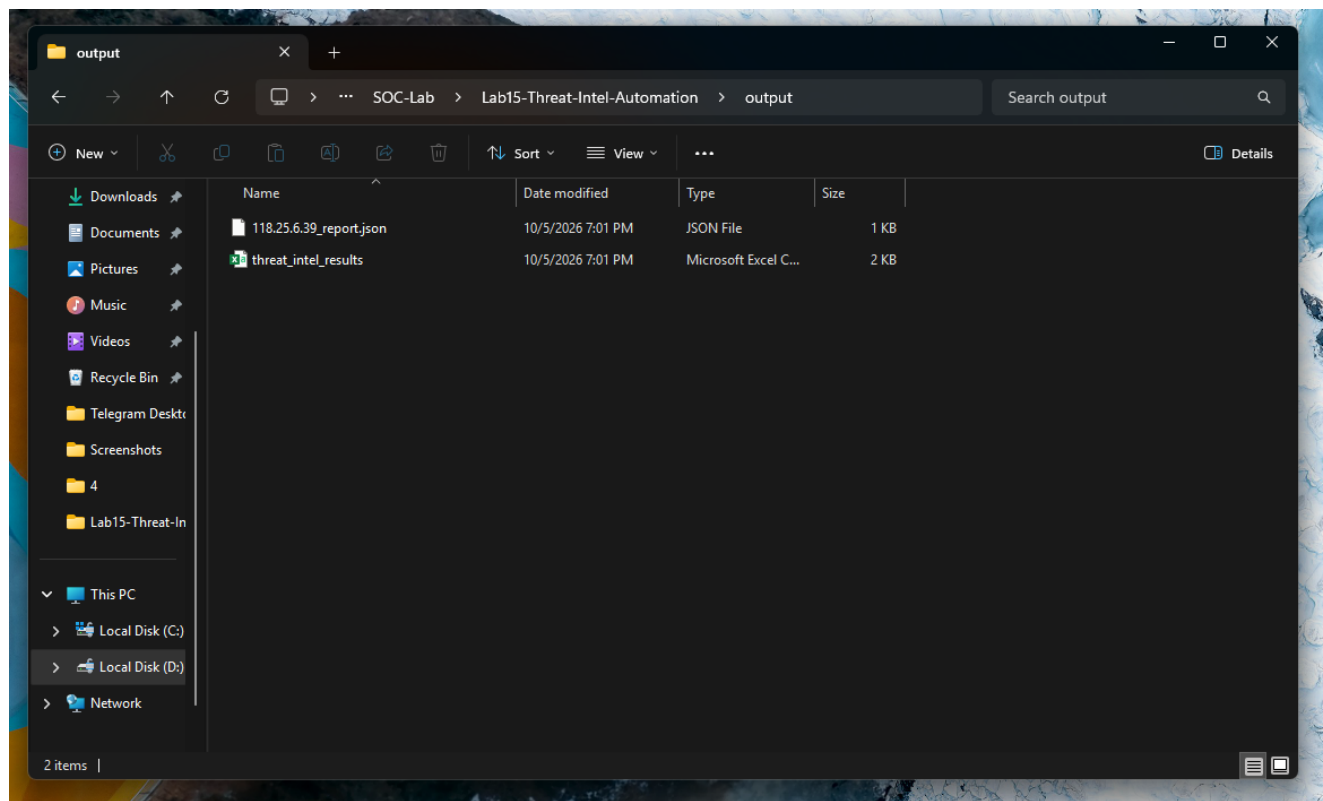
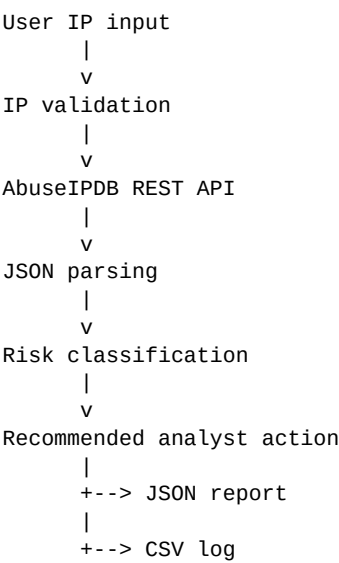


Figure 5 - Output folder containing the generated JSON report and cumulative CSV results file.

Output	Purpose
118.25.6.39_report.json	Structured report for the individual IP lookup
threat_intel_results.csv	Cumulative CSV log for repeated lookups and analyst review

Automation Flow



Security Engineering Assessment

Capability	Assessment
Credential handling	API key stored outside source code using an environment variable
Input validation	Invalid and private/reserved addresses rejected before API use
REST API integration	Successful authenticated AbuseIPDB HTTPS request
JSON parsing	Relevant reputation fields extracted from the API response
Automated decision logic	Lab-defined LOW/MEDIUM/HIGH classification and recommended action
Structured output	Per-IP JSON report and cumulative CSV logging
Error handling	Request and missing-field errors handled with try/except

Security Engineering Lessons

- Security automation scripts do not need to be large to provide operational value; reliable validation, API handling, and structured output are key.
- Credentials should be separated from source code and excluded from screenshots and public repositories.
- REST API responses should be parsed into concise analyst-facing fields instead of relying only on raw JSON.
- Risk scores are enrichment context and should not be treated as proof that an IP is malicious.
- Input validation reduces unnecessary API calls and helps prevent invalid data from entering downstream workflows.
- JSON and CSV outputs make a small Python script easier to integrate with SIEM, SOAR, reporting, or future cloud-security workflows.

Final Conclusion

Lab 15 successfully demonstrated a practical Python threat-intelligence automation workflow. The tool validates public IP input, securely accesses the AbuseIPDB API, parses reputation data, assigns a lab-defined risk level, generates a recommended analyst action, and stores results as both JSON and CSV. The lab provides hands-on evidence of Python scripting, REST API integration, data processing, secure credential handling, input validation, error handling, and security workflow automation.

Safety note: This was a controlled portfolio lab using a public test IP and threat-intelligence data. No systems were scanned or attacked, and the AbuseIPDB API key is not included in this report, screenshots, or source examples.